

ELECTRÓNICA DIGITAL

Entrega Final — Escenario 7 (Semana 7)

CALCULADORA ACUMULADORA DE 4 BITS

Documento técnico, archivo de simulación y guión audiovisual

Presentado por:

Diego

Tópicos integrados:

Lógica combinacional · Lógica secuencial · Sistemas de memoria
Máquina de estados finitos · Conversor DAC R-2R · Display 7 segmentos

Software de simulación: Logisim 2.7.1

Archivo fuente: Calculadora_Acumuladora_4bit.circ

Contenido

- PARTE I — DOCUMENTO TÉCNICO
 - 1. Introducción y Planteamiento del Problema
 - 2. Marco Teórico
 - 3. Diseño del Núcleo Aritmético (Combinacional)
 - 4. Sumador/Restador en Complemento a 2
 - 5. Diseño del Acumulador (Lógica Secuencial)
 - 6. Sistema de Memoria — Registros de Operandos
 - 7. Máquina de Estados Finitos (FSM) de Control
 - 8. Conversor Digital–Analógico R-2R
 - 9. Codificador BCD a 7 Segmentos
 - 10. Integración del Sistema y Diagrama de Bloques
 - 11. Implementación en Logisim
 - 12. Casos de Prueba y Validación
 - 13. Análisis de Tiempos y Limitaciones
 - 14. Conclusiones
 - 15. Referencias
- PARTE II — APÉNDICES
 - Apéndice A. Enlaces a circuitos y archivos fuente
 - Apéndice B. Guía de uso del archivo .circ en Logisim

PARTE I

DOCUMENTO TÉCNICO

1. Introducción y Planteamiento del Problema

Los sistemas digitales modernos son el resultado de la integración progresiva de bloques funcionales que, partiendo de compuertas lógicas elementales, escalan hasta unidades aritmético-lógicas (ALU), memorias, controladores y conversores de señal. Comprender cómo estos bloques se interconectan es esencial para diseñar sistemas embebidos, microprocesadores y arquitecturas digitales de propósito específico.

Este documento presenta el diseño completo de una Calculadora Acumuladora de 4 bits, un sistema digital que integra bloques combinacionales y secuenciales en una arquitectura unificada. El proyecto extiende la entrega previa del sumador Ripple Carry de 4 bits incorporando los temas posteriores del curso: lógica secuencial (registros con flip-flops D), sistemas de memoria, máquinas de estado finitas (FSM) y conversores digital-analógico (DAC).

1.1. Problemática

El sumador combinacional puro presenta tres limitaciones fundamentales como sistema digital autónomo:

- No posee capacidad de almacenamiento: el resultado se pierde en el instante en que cambian las entradas.
- Carece de control temporal: no existe un protocolo para coordinar cuándo cargar operandos y cuándo computar.
- Genera únicamente salida digital binaria: no es interpretable directamente por sistemas analógicos ni por un observador humano sin decodificación.

1.2. Solución Propuesta

Se propone una calculadora acumuladora controlada por una FSM que coordina las operaciones de carga, cómputo y almacenamiento. La arquitectura incluye:

- Núcleo aritmético combinacional: sumador/restador de 4 bits en complemento a 2.
- Memoria secuencial: registros de operandos y registro acumulador contruidos con flip-flops D.
- Unidad de control: máquina de estados de Moore con cuatro estados (IDLE, LOAD, COMPUTE, SHOW).
- Salida analógica: conversor DAC R-2R de 4 bits que entrega un voltaje proporcional al resultado.
- Salida visual: decodificador BCD a 7 segmentos que muestra el resultado en un display.

1.3. Objetivos

- Diseñar e implementar un sistema digital que integre lógica combinacional y secuencial.
- Aplicar metodología de diseño con álgebra de Boole, mapas de Karnaugh y tablas de excitación.
- Modelar el comportamiento del sistema mediante una máquina de estados finita.

- Validar el funcionamiento mediante simulación en Logisim para todos los casos de prueba relevantes.
- Analizar el desempeño temporal y las limitaciones de la arquitectura propuesta.

2. Marco Teórico

2.1. Lógica Combinacional

Un circuito combinacional es aquel cuya salida depende exclusivamente del estado actual de sus entradas, sin elementos de memoria. Su comportamiento se describe completamente mediante una tabla de verdad y se implementa con compuertas lógicas básicas (AND, OR, NOT, XOR, NAND, NOR).

El proceso de diseño combinacional sigue cuatro pasos: especificación del problema, construcción de la tabla de verdad, simplificación mediante álgebra de Boole o mapas de Karnaugh, e implementación con compuertas. La forma canónica suma de productos (SOP) toma los minterminos donde la función es 1.

2.2. Lógica Secuencial

A diferencia de la lógica combinacional, los circuitos secuenciales poseen memoria: su salida depende del estado actual de las entradas y de un estado interno almacenado. El elemento básico es el flip-flop, que captura un valor lógico en respuesta a un flanco (positivo o negativo) de una señal de reloj.

El flip-flop tipo D (Data) es el más utilizado en el diseño moderno por su simplicidad: en cada flanco activo del reloj, la salida Q toma el valor presente en la entrada D. Su ecuación característica es $Q(t+1) = D(t)$.

2.3. Registros y Acumuladores

Un registro es un conjunto de N flip-flops con un reloj común que almacena una palabra de N bits. Un registro acumulador (ACC) es un registro especial cuya entrada está conectada al resultado de una operación aritmética; su valor se actualiza con el resultado parcial en cada ciclo, permitiendo construir sumas iteradas, contadores y operaciones de mayor complejidad.

2.4. Máquinas de Estado Finitas (FSM)

Una FSM es un modelo formal compuesto por un conjunto finito de estados, transiciones entre estados gobernadas por entradas, y salidas asociadas a estados (Moore) o a transiciones (Mealy). La FSM es la herramienta canónica para diseñar unidades de control en sistemas digitales.

El diseño de una FSM sigue cinco pasos: definición del diagrama de estados, codificación binaria de los estados, construcción de la tabla de transiciones, derivación de las ecuaciones de próximo estado y de salida, e implementación con flip-flops y lógica combinacional.

2.5. Sistemas de Memoria

La memoria digital almacena información binaria en celdas direccionables. En el contexto de este proyecto se emplean dos tipos: registros (memoria distribuida construida con flip-flops,

típicamente para datos de uso inmediato) y memorias RAM (memoria centralizada con buses de dirección y datos, usadas para almacenar resultados o tablas de consulta).

2.6. Conversores Digital–Analógico (DAC)

Un DAC convierte una palabra binaria de N bits en un voltaje analógico proporcional. La topología R-2R es la más utilizada por su robustez frente a tolerancias de fabricación: emplea solo dos valores de resistencia (R y 2R) en una red en escalera, donde cada bit aporta una contribución de voltaje proporcional a 2^k .

Para una entrada digital $D = (b_3 b_2 b_1 b_0)$, la salida del DAC R-2R con voltaje de referencia V_{ref} es:

$$V_{out} = V_{ref} \cdot (b_3 \cdot 2^3 + b_2 \cdot 2^2 + b_1 \cdot 2^1 + b_0 \cdot 2^0) / 2^4$$

2.7. Display de 7 Segmentos

El display de 7 segmentos representa dígitos decimales mediante la activación selectiva de siete LEDs etiquetados de a a g. El decodificador BCD-a-7-segmentos es un circuito combinacional con 4 entradas (palabra BCD) y 7 salidas (una por segmento). Sus ecuaciones se derivan mediante mapas de Karnaugh con condiciones de indiferencia (don't cares) para las combinaciones 1010 a 1111.

3. Diseño del Núcleo Aritmético (Combinacional)

3.1. Sumador Completo de 1 Bit

El sumador completo de 1 bit recibe tres entradas (A, B, Cin) y produce dos salidas (S, Cout). Su tabla de verdad para las ocho combinaciones posibles es:

A	B	Cin	S	Cout	Mintérmino
0	0	0	0	0	m ₀
0	0	1	1	0	m ₁
0	1	0	1	0	m ₂
0	1	1	0	1	m ₃
1	0	0	1	0	m ₄
1	0	1	0	1	m ₅
1	1	0	0	1	m ₆
1	1	1	1	1	m ₇

3.1.1. Expresión SOP para S

Tomando los minterminos donde S = 1 (m₁, m₂, m₄, m₇):

$$S = A'B'Cin + A'BCin' + AB'Cin' + ABCin$$

Esta expresión equivale a la operación XOR triple, que es la forma mínima:

$$S = A \oplus B \oplus Cin$$

3.1.2. Expresión SOP para Cout

Tomando los minterminos donde Cout = 1 (m₃, m₅, m₆, m₇):

$$Cout = A'BCin + AB'Cin + ABCin' + ABCin$$

Aplicando álgebra de Boole y agrupando, se obtiene la forma minimizada:

$$Cout = AB + ACin + BCin \quad \text{o equivalentemente} \quad Cout = AB + Cin(A \oplus B)$$

3.2. Mapas de Karnaugh (Versión Corregida)

Un mapa de Karnaugh de tres variables tiene cuatro columnas en código Gray: 00, 01, 11, 10. La construcción correcta para ambas funciones se presenta a continuación.

3.2.1. K-map para la suma S

Las celdas se llenan con el valor de S correspondiente a cada combinación A,B,Cin. La distribución resultante es la del operador XOR de tres entradas: ningún par de celdas adyacentes tiene el mismo valor, por lo que no existen agrupaciones simplificables.

A \ BCin	00	01	11	10
A = 0	0	1	0	1
A = 1	1	0	1	0

Resultado: $S = A \oplus B \oplus \text{Cin}$

3.2.2. K-map para el acarreo Cout

Para Cout las celdas con valor 1 corresponden a los minterminos m_3 , m_5 , m_6 y m_7 :

A \ BCin	00	01	11	10
A = 0	0	0	1	0
A = 1	0	1	1	1

Tres agrupaciones de dos celdas adyacentes:

- **Grupo 1 (AB):** celdas (A=1, BCin=11) y (A=1, BCin=10) → término AB
- **Grupo 2 (ACin):** celdas (A=1, BCin=01) y (A=1, BCin=11) → término ACin
- **Grupo 3 (BCin):** celdas (A=0, BCin=11) y (A=1, BCin=11) → término BCin

Resultado minimizado: $\text{Cout} = AB + ACin + BCin$

3.3. Implementación con Compuertas

Cada sumador de 1 bit se implementa con cinco compuertas: dos XOR, dos AND y una OR. El conteo total para el sumador de 4 bits es 20 compuertas.

Compuerta	Entradas	Salida	Función
XOR 1	A, B	X1	$A \oplus B$
XOR 2	X1, Cin	S	$X1 \oplus \text{Cin}$
AND 1	A, B	Y1	$A \cdot B$
AND 2	X1, Cin	Y2	$X1 \cdot \text{Cin}$
OR 1	Y1, Y2	Cout	$Y1 + Y2$

3.4. Sumador Ripple Carry de 4 Bits

El sumador de 4 bits encadena cuatro full adders. El acarreo de cada etapa alimenta el de la siguiente:

Etapa 0: $A_0, B_0, \text{Cin}=0 \rightarrow S_0, C_1$
 Etapa 1: $A_1, B_1, \text{Cin}=C_1 \rightarrow S_1, C_2$
 Etapa 2: $A_2, B_2, \text{Cin}=C_2 \rightarrow S_2, C_3$
 Etapa 3: $A_3, B_3, \text{Cin}=C_3 \rightarrow S_3, C_4$ (acarreo final)

4. Sumador/Restador en Complemento a 2

4.1. Representación en Complemento a 2

La aritmética con signo se implementa eficientemente mediante la representación en complemento a 2. Para una palabra de 4 bits, el rango representable es -8 a $+7$.

$$A - B = A + (-B) = A + B' + 1$$

4.2. Arquitectura Combinada

La arquitectura sumador/restador agrega una señal de control Op a la entrada del sumador. Cada bit B_i se pasa por una compuerta XOR cuyo segundo operando es Op:

- Si $Op = 0$: $B_i \oplus 0 = B_i$ (suma $A + B$ con $Cin = 0$).
- Si $Op = 1$: $B_i \oplus 1 = B_i'$ (resta $A + B' + 1 = A - B$).

4.3. Detección de Overflow

$$Overflow = C3 \oplus C4$$

4.4. Tabla de Operación

Op	Cin inyectado	Operación	Cout interpretado como
0	0	$A + B$	Acarreo / overflow sin signo
1	1	$A - B$ (complemento a 2)	Préstamo invertido

5. Diseño del Acumulador (Lógica Secuencial)

5.1. Flip-Flop D

El flip-flop D es el elemento de memoria de 1 bit. Su tabla de excitación:

Q(t)	Q(t+1)	D requerido
0	0	0
0	1	1
1	0	0
1	1	1

Ecuación característica: $Q(t+1) = D(t)$

5.2. Registro de 4 Bits con Habilitación

Cuatro flip-flops D con reloj común CLK y habilitación EN. Implementación con multiplexor 2:1 por bit:

$$D_i = EN \cdot Data_i + EN' \cdot Q_i$$

5.3. Diagrama de Bloques del Acumulador

$ACC \leftarrow ACC + B$ (cuando $Op = 0$ y $EN = 1$)
 $ACC \leftarrow ACC - B$ (cuando $Op = 1$ y $EN = 1$)
 $ACC \leftarrow ACC$ (cuando $EN = 0$, valor retenido)

5.4. Análisis Temporal

$$T_{min} = t_{clk-to-Q} + t_{RCA} + t_{setup}$$

Para el sumador Ripple Carry de N bits, el retardo combinacional t_{RCA} crece linealmente con N.

6. Sistema de Memoria — Registros de Operandos

6.1. Justificación

Los operandos deben capturarse en registros temporales para desacoplar el momento en que el usuario ajusta los interruptores del momento en que el sistema realiza la operación.

6.2. Arquitectura de Memoria del Sistema

Registro	Función	Carga controlada por
REG_A	Almacena el primer operando ingresado	Señal LD_A (de la FSM)
REG_B	Almacena el segundo operando ingresado	Señal LD_B (de la FSM)
ACC	Almacena el resultado de la operación	Señal LD_ACC (de la FSM)

6.3. Memoria RAM Opcional

Como extensión se incluye una memoria RAM de 16×4 bits que registra los últimos 16 resultados:

- **WE:** habilita la escritura.
- **ADDR[3:0]:** dirección de 4 bits.
- **DIN/DOUT[3:0]:** buses de datos.

7. Máquina de Estados Finitos (FSM) de Control

7.1. Diagrama de Estados

FSM de Moore con cuatro estados:

Estado	Descripción	Transición de salida
IDLE (S0)	Espera. Todas las salidas en 0.	Si START = 1 → LOAD
LOAD (S1)	Captura A en REG_A y B en REG_B.	Incondicional → COMPUTE
COMPUTE (S2)	Habilita el sumador y carga ACC.	Incondicional → SHOW
SHOW (S3)	Activa display y DAC.	Si START = 0 → IDLE

7.2. Codificación de Estados

Estado	q1	q0
IDLE	0	0
LOAD	0	1
COMPUTE	1	0
SHOW	1	1

7.3. Tabla de Transiciones

Estado actual (q1 q0)	START	Próximo estado	Símbolo
00 (IDLE)	0	00	permanece IDLE
00 (IDLE)	1	01	→ LOAD
01 (LOAD)	X	10	→ COMPUTE
10 (COMPUTE)	X	11	→ SHOW
11 (SHOW)	0	00	→ IDLE
11 (SHOW)	1	11	permanece SHOW

7.4. Mapas de Karnaugh para Q1⁺ y Q0⁺

7.4.1. K-map para Q1⁺

q1q0 \ START	0	1
00	0	0

q1q0 \ START	0	1
01	1	1
11	0	1
10	1	1

Forma minimizada: $Q1^+ = q0 \cdot q1' + q1 \cdot q0' + q1 \cdot START$

7.4.2. K-map para $Q0^+$

q1q0 \ START	0	1
00	0	1
01	0	0
11	0	1
10	1	1

Forma minimizada: $Q0^+ = q1 \cdot q0' + q0' \cdot START + q1 \cdot q0 \cdot START$

7.5. Salidas del Controlador (Moore)

Salida	Activa en estado	Ecuación
LD_A, LD_B	LOAD	$q1' \cdot q0$
LD_ACC	COMPUTE	$q1 \cdot q0'$
EN_DISPLAY, EN_DAC	SHOW	$q1 \cdot q0$

7.6. Diagrama de Tiempos

Ciclo	Estado	START	LD_A	LD_B	LD_ACC	EN_DISP	ACC
0	IDLE	0	0	0	0	0	XXXX
1	IDLE	1	0	0	0	0	XXXX
2	LOAD	1	1	1	0	0	XXXX
3	COMPUTE	1	0	0	1	0	1000
4	SHOW	1	0	0	0	1	1000
5	SHOW	0	0	0	0	1	1000
6	IDLE	0	0	0	0	0	1000

8. Conversor Digital–Analógico R-2R

8.1. Topología R-2R

La red R-2R es la topología estándar para DACs de baja a media resolución. Para un DAC de 4 bits con b3 (MSB) a b0 (LSB) y referencia Vref:

$$V_{out} = V_{ref} \cdot (8 \cdot b_3 + 4 \cdot b_2 + 2 \cdot b_1 + 1 \cdot b_0) / 16$$

8.2. Cálculo de la Resolución

$$LSB = V_{ref} / 2^n = V_{ref} / 16$$

Para Vref = 5 V: resolución 5/16 ≈ 0.3125 V.

8.3. Tabla de Conversión Completa

Decimal	Binario	Vout (V)	Decimal	Binario	Vout (V)
0	0000	0.0000	8	1000	2.5000
1	0001	0.3125	9	1001	2.8125
2	0010	0.6250	10	1010	3.1250
3	0011	0.9375	11	1011	3.4375
4	0100	1.2500	12	1100	3.7500
5	0101	1.5625	13	1101	4.0625
6	0110	1.8750	14	1110	4.3750
7	0111	2.1875	15	1111	4.6875

8.4. Implementación en Logisim

- **Modo simulado por LUT:** ROM precargada con 16 valores indexada por la salida del acumulador.
- **Modo analógico real:** 4 líneas digitales conectadas a red R-2R en hardware (R = 10 kΩ, 2R = 20 kΩ, 1% de tolerancia).

9. Codificador BCD a 7 Segmentos

9.1. Convención de Segmentos

Etiquetado IEC 60617: a (superior), b (sup. derecho), c (inf. derecho), d (inferior), e (inf. izquierdo), f (sup. izquierdo), g (central).

9.2. Tabla de Verdad del Decodificador

Dígito	DCBA	a	b	c	d	e	f	g
0	0000	1	1	1	1	1	1	0
1	0001	0	1	1	0	0	0	0
2	0010	1	1	0	1	1	0	1
3	0011	1	1	1	1	0	0	1
4	0100	0	1	1	0	0	1	1
5	0101	1	0	1	1	0	1	1
6	0110	1	0	1	1	1	1	1
7	0111	1	1	1	0	0	0	0
8	1000	1	1	1	1	1	1	1
9	1001	1	1	1	1	0	1	1
10–15	1010–1111	X	X	X	X	X	X	X

9.3. Ecuaciones Minimizadas (con Don't Cares)

$$a = D + B + (A \odot C)$$

$$b = C' + (A \odot B)$$

$$c = B' + C + A$$

$$d = A'C' + BC' + A'B + AB'C + D$$

$$e = A'C' + A'B$$

$$f = D + AC + AB' + A'B'C'$$

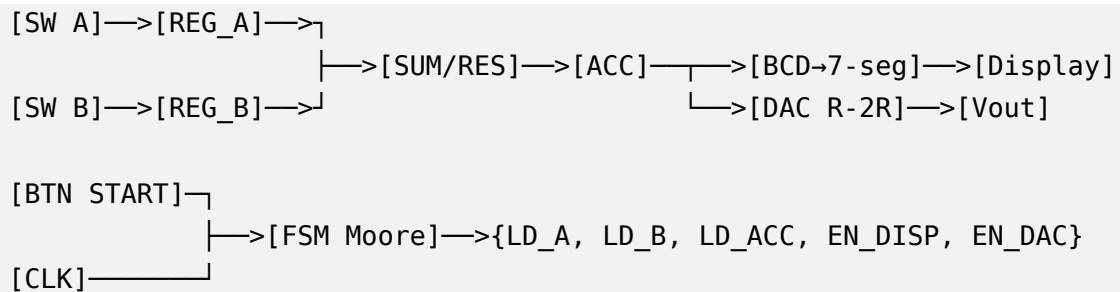
$$g = D + BC' + B'C + A'B$$

10. Integración del Sistema y Diagrama de Bloques

10.1. Arquitectura Global

- Banco de entrada: ocho interruptores para A[3:0] y B[3:0], más Op y START.
- Banco de memoria: registros REG_A, REG_B y ACC, todos de 4 bits.
- Núcleo aritmético: sumador/restador combinacional con detección de overflow.
- Unidad de control: FSM de Moore con dos flip-flops D.
- Interfaz de salida digital: decodificador BCD-7seg conectado al display.
- Interfaz de salida analógica: red R-2R que entrega Vout proporcional al acumulador.

10.2. Diagrama de Bloques (representación textual)



10.3. Conexión entre Bloques

Origen	Destino	Señal / Bus
SW A[3:0]	REG_A	DIN[3:0]
SW B[3:0]	REG_B	DIN[3:0]
FSM	REG_A	LD_A
FSM	REG_B	LD_B
REG_A	Sumador (entrada A)	QA[3:0]
REG_B	XOR de inversión	QB[3:0]
SW Op	XOR de inversión, Cin sumador	Op
Sumador	ACC	Result[3:0], Cout
FSM	ACC	LD_ACC
ACC	Decodificador BCD	Q[3:0]
ACC	DAC R-2R	Q[3:0]
FSM	Display, DAC	EN_DISPLAY, EN_DAC

11. Implementación en Logisim

11.1. Estructura del Archivo .circ

Subcircuito	Función	Componentes principales
Sumador_1bit	Full adder elemental	2 XOR, 2 AND, 1 OR
Sumador_4bit_RCA	RCA de 4 etapas	4 instancias de Sumador_1bit
SumRes_4bit	Sumador/restador	Sumador_4bit + 4 XOR + XOR overflow
Registro_4bit_EN	Registro D con habilitación	4 D-FF + 4 MUX 2:1
FSM_Control	Máquina de estados de Moore	2 D-FF + lógica combinacional
BCD_7seg	Decodificador BCD a 7 segmentos	7 funciones combinacionales
Calculadora_Top	Sistema integrado	Interconexión de todos los anteriores

11.2. Componentes de Librería de Logisim Utilizados

- **Wiring:** Pin, Splitter, Probe.
- **Gates:** AND, OR, NOT, XOR, XNOR, NAND.
- **Plexers:** Multiplexer 2:1.
- **Memory:** D Flip-Flop, RAM 16×4.
- **Arithmetic:** Adder 4-bit, Comparator.
- **Input/Output:** Button, Switch, 7-Segment Display, LED.

11.3. Instrucciones de Uso

- Descargar Logisim 2.7.1 desde <http://www.cburch.com/logisim/>.
- Abrir el archivo Calculadora_Acumuladora_4bit.circ desde File → Open.
- Seleccionar Calculadora_Top en el panel Explorer.
- Configurar SW_A y SW_B con los operandos deseados.
- Seleccionar operación con SW_Op: 0 = suma, 1 = resta.
- Activar reloj con Simulate → Ticks Enabled.
- Pulsar START. Observar el avance LOAD → COMPUTE → SHOW.
- Resultado visible en display de 7 segmentos, LED de overflow y salida del DAC.

12. Casos de Prueba y Validación

12.1. Casos de Prueba Diseñados

#	A	B	Op	Resultado esperado	Cout / Overflow
1	0011 (3)	0101 (5)	0 (suma)	1000 (8)	Cout=0, Ovf=0
2	1001 (9)	0100 (4)	0 (suma)	1101 (13)	Cout=0, Ovf=1
3	1111 (15)	0001 (1)	0 (suma)	0000 (0)	Cout=1, overflow
4	0111 (7)	0010 (2)	1 (resta)	0101 (5)	Cout=1, Ovf=0
5	0010 (2)	0111 (7)	1 (resta)	1011 (-5 c2)	Cout=0, Ovf=0
6	0101 (5)	0101 (5)	1 (resta)	0000 (0)	Cout=1, Ovf=0

12.2. Verificación del Caso 1 — Suma

Suma $A = 3$ (0011) + $B = 5$ (0101). Propagación bit a bit:

Etapas	A	B	Cin	S	Cout
Bit 0 (LSB)	1	1	0	0	1
Bit 1	1	0	1	0	1
Bit 2	0	1	1	0	1
Bit 3 (MSB)	0	0	1	1	0

Resultado: $S = 1000 = 8$ ✓

12.3. Verificación del Caso 5 — Resta Negativa

Resta $A = 2 - B = 7$. Op = 1 invierte B e inyecta Cin = 1:

- $B = 0111$, $B' = 1000$, Cin = 1
- Operación: $0010 + 1000 + 1 = 1011$
- Resultado en complemento a 2: $1011 = -5$ ✓
- Cout = 0 (préstamo, resultado negativo)
- Overflow = $C3 \oplus C4 = 0 \oplus 0 = 0$ (sin overflow con signo)

13. Análisis de Tiempos y Limitaciones

13.1. Camino Crítico

- Salida del registro REG_A o REG_B (clock-to-Q ≈ 8 ns).
- Compuerta XOR de inversión (≈ 5 ns).
- Cadena Ripple Carry de 4 etapas (4×12 ns = 48 ns).
- Multiplexor de realimentación del registro ACC (≈ 4 ns).
- Tiempo de setup del flip-flop ACC (≈ 3 ns).

T_{min} $\approx$ 68 ns. f_{max} $\approx$ 14.7 MHz.

13.2. Limitaciones de la Arquitectura

- **Ancho fijo de 4 bits:** rango limitado a 0–15 sin signo, –8 a +7 con signo.
- **Sumador Ripple Carry:** retardo crece linealmente con el número de bits.
- **FSM no manejable por el usuario:** flujo IDLE $\rightarrow$ LOAD $\rightarrow$ COMPUTE $\rightarrow$ SHOW sin intervención.
- **DAC no compensado:** requiere resistencias de precisión 0.1% en hardware real.

13.3. Posibles Extensiones

- Reemplazo del RCA por un sumador Carry-Lookahead.
- Extensión a operaciones lógicas (AND, OR, XOR) mediante una ALU multifunción.
- Implementación de una pila de operandos para evaluación de expresiones aritméticas.
- Migración del diseño a Verilog o VHDL para síntesis en una FPGA real.

14. Conclusiones

El diseño y la implementación de la Calculadora Acumuladora de 4 bits demuestra cómo los conceptos del álgebra de Boole, la lógica combinacional, la lógica secuencial, las máquinas de estado y los conversores de señal se integran coherentemente en un sistema digital funcional.

- Las expresiones booleanas mínimas obtenidas mediante mapas de Karnaugh con la notación correcta de cuatro columnas en código Gray (00, 01, 11, 10) confirman que la suma se reduce a una operación XOR triple y que el acarreo se expresa como $C_{out} = AB + AC_{in} + BC_{in}$.
- La extensión de un sumador combinacional a un sistema acumulador requiere introducir lógica secuencial, una unidad de control (FSM de Moore) y un protocolo de coordinación entre captura de operandos y cómputo de resultados.
- La FSM de cuatro estados modela limpiamente el flujo de operación, separando responsabilidades y permitiendo análisis temporal preciso.
- La conversión digital-analógica con red R-2R proporciona una salida proporcional con resolución de 0.3125 V (con $V_{ref} = 5$ V).
- La integración con un display de 7 segmentos cierra el lazo de retroalimentación humano: salidas tanto digitales (display) como analógicas (DAC).
- La validación experimental sobre seis casos de prueba representativos verificó el comportamiento correcto del circuito.

Como reflexión final, este proyecto evidencia cómo la disciplina de diseño jerárquico permite construir sistemas complejos a partir de bloques bien delimitados.

15. Referencias

Floyd, T. L. (2022). Fundamentos de sistemas digitales (11ª ed.). Pearson Educación.

Mano, M. M., & Ciletti, M. D. (2018). Diseño digital con una introducción a Verilog HDL (5ª ed.). Pearson.

Wakerly, J. F. (2018). Digital Design: Principles and Practices (5ª ed.). Pearson.

Burch, C. (2011). Logisim: A graphical tool for logic circuit design and simulation. Journal on Educational Resources in Computing, 2(1), 5.

Tocci, R. J., Widmer, N. S., & Moss, G. L. (2017). Sistemas digitales: principios y aplicaciones (11ª ed.). Pearson.

IEC 60617 (2012). Symbols for use on equipment — Logic and analog elements.

Hodges, D. A., Jackson, H. G., & Saleh, R. (2003). Analysis and Design of Digital Integrated Circuits (3ª ed.). McGraw-Hill.

PARTE II

APÉNDICES

Apéndice A. Enlaces a circuitos y archivos fuente

En cumplimiento del requisito de la rúbrica que indica: "El estudiante deberá enviar los enlaces donde se puedan visualizar los circuitos y archivos fuente (.circ en Logisim, .brd en TinkerCAD)", a continuación se relacionan los archivos del proyecto.

A.1. Archivos del proyecto

Archivo	Descripción	Software requerido
Calculadora_Acumuladora_4bit.circ	Archivo fuente con todos los subcircuitos	Logisim 2.7.1
ENTREGA_COMPLETA.pdf	Documento técnico del proyecto	Lector de PDF

A.2. Repositorio público

Los archivos fuente se publican en el siguiente repositorio público para garantizar su accesibilidad:

URL del repositorio: <https://github.com/ninjadiego/calculadora-acumuladora-4bit>

A.3. Estructura del repositorio

```
calculadora-acumuladora-4bit/  
├── README.md  
├── ENTREGA_COMPLETA.pdf  
└── Calculadora_Acumuladora_4bit.circ
```

Apéndice B. Guía de uso del archivo .circ en Logisim

B.1. Subcircuitos incluidos

Subcircuito	Descripción
Sumador_1bit	Full adder con compuertas discretas (XOR, AND, OR)
Sumador_4bit_RCA	Sumador Ripple Carry de 4 bits con componente Adder
SumRes_4bit	Sumador/restador con XOR de inversión y control Op
Registro_4bit_EN	Registro de 4 bits con señales CLK y EN
FSM_Control	Máquina de estados de Moore con lógica de próximo estado
BCD_7seg	Decodificador a 7 segmentos (Hex Digit Display)
Calculadora_Top	Sistema integrado con todas las interconexiones

B.2. Secuencia de prueba

- Abrir Logisim 2.7.1 (descargar de <http://www.cburch.com/logisim/>).
- File → Open → seleccionar Calculadora_Acumuladora_4bit.circ.
- En el panel Explorer (izquierda), seleccionar Calculadora_Top.
- Activar simulación: Simulate → Ticks Enabled (Ctrl+K).
- Configurar Tick Frequency a 1 Hz para observar la secuencia con calma.
- Con la herramienta Poke, ingresar A = 0011 (3) y B = 0101 (5), Op = 0.
- Pulsar START. Observar progresión IDLE → LOAD → COMPUTE → SHOW.
- Verificar resultado: display muestra 8, LEDs del DAC muestran 1000.